

Servicio Nacional de Aprendizaje SENA
Centro de Gestión de Mercados, Logística y Tecnologías de la Información
Tecnólogo en Análisis y Desarrollo de Software

Evidencia. GA5-220501095-AA1-EV02

Nociones de Reglas de Usabilidad y Accesibilidad en Aplicaciones Independientes
TRUEQUE MATCH (Aplicación de Intercambio Colaborativo)

Presentado por:

Gerson Jonnathan López Oviedo

Código de aprendiz: 3186647

Docente:

Arnaldo Alfonso Montiel Brun

Fase 2. Planeación Actividad de Proyecto 5

Bogotá D.C., 2025

ÍNDICE

Introducción	3
Sección 1. Usabilidad en el Desarrollo de Software	4
1.1 ¿Qué es la usabilidad?	4
1.2 Componentes de la Usabilidad:	4
1.3 Las 10 Heurísticas de Usabilidad de Nielsen:	5
1.4 Reglas de Usabilidad de Shneiderman	7
Sección 2. Accesibilidad en Aplicaciones de Software	9
2.1 ¿Qué es la accesibilidad?	9
2.2 Pautas WCAG 2.1 El Estándar Internacional.....	9
2.3 Niveles de Conformidad WCAG:	10
2.4 Reglas de Accesibilidad Aplicadas a TRUEQUE MATCH:	11
Sección 3. Tipos de Aplicaciones: Independiente vs. Web	13
3.1 Aplicación Independiente (Desktop / Nativa)	13
3.2 Aplicación Web:.....	15
3.3 Comparación: Aplicación Independiente vs. Aplicación Web.....	17
3.4 Tipo de Aplicación de TRUEQUE MATCH	18
Conclusiones	20
Referencias.....	21

Introducción

Este documento corresponde a la evidencia GA5-220501095-AA1-EV02 del programa Tecnólogo en Análisis y Desarrollo de Software del SENA. Su propósito es presentar los conceptos fundamentales de usabilidad y accesibilidad en el desarrollo de aplicaciones de software, diferenciando entre aplicaciones independientes y aplicaciones web, y describiendo las reglas que guían el diseño de interfaces de calidad.

Estos conceptos se aplican directamente al proyecto TRUEQUE MATCH, una aplicación híbrida de intercambio colaborativo sin dinero, para mostrar cómo las reglas de usabilidad y accesibilidad se convierten en decisiones concretas de diseño que mejoran la experiencia del usuario colombiano.

El documento está organizado en tres secciones: fundamentos de usabilidad con sus reglas y principios, fundamentos de accesibilidad con sus pautas internacionales, y la diferenciación entre aplicaciones independientes y aplicaciones web con sus características propias.

Sección 1. Usabilidad en el Desarrollo de Software

1.1 ¿Qué es la usabilidad?

La usabilidad es la capacidad de un sistema de software para ser usado por personas específicas, en un contexto determinado, para alcanzar sus objetivos de manera eficaz, eficiente y con satisfacción (ISO 9241-11, 2018). En términos directos: una app es usable cuando el usuario logra lo que necesita sin complicarse la vida.

Nielsen (1993) define la usabilidad como un atributo de calidad que mide qué tan fácil es usar una interfaz. No se trata solo de que la app sea bonita, sino de que funcione bien para quien la usa. Una app puede verse increíble y al mismo tiempo ser un caos para navegar, y eso la hace poco usable.

Para TRUEQUE MATCH, esto se traduce en que un usuario mayor de 18 años, sin formación técnica, debe poder registrarse, publicar una oferta y proponer un trueque en menos de 5 minutos, sin necesitar manual ni ayuda.

1.2 Componentes de la Usabilidad:

Nielsen (1993) establece cinco componentes que definen la usabilidad de forma medible:

Componente	Definición	Ejemplo en TRUEQUE MATCH
FACILIDAD DE APRENDIZAJE	¿QUÉ TAN RÁPIDO APRENDE EL USUARIO A USAR EL SISTEMA LA PRIMERA VEZ?	EL REGISTRO GUIADO EN 3 PASOS PERMITE COMPLETAR LA CUENTA SIN INSTRUCCIONES PREVIAS.
EFICIENCIA	UNA VEZ APRENDIDO, ¿QUÉ TAN RÁPIDO PUEDE REALIZAR LAS TAREAS?	LA TAB BAR INFERIOR PERMITE ACCEDER A CUALQUIER SECCIÓN PRINCIPAL EN UN SOLO TOQUE.
FACILIDAD DE MEMORIZACIÓN	SI EL USUARIO NO USA LA APP POR UN TIEMPO, ¿PUEDE VOLVER SIN REAPRENDER?	LOS ÍCONOS ESTÁNDAR (CASA, LUPA, MÁS, CHAT, PERFIL) SON RECONOCIBLES UNIVERSALMENTE.
TOLERANCIA A ERRORES	¿CUÁNTOS ERRORES COMETE EL USUARIO Y QUÉ TAN FÁCIL LOS RECUPERA?	LOS MENSAJES DE ERROR BAJO CADA CAMPO EXPLICAN QUÉ FALLÓ Y CÓMO CORREGIRLO.
SATISFACCIÓN SUBJETIVA	¿LE GUSTA USAR EL SISTEMA AL USUARIO?	DISEÑO VISUAL CONSISTENTE CON IDENTIDAD PROPIA QUE GENERA CONFIANZA Y AGRADO.

1.3 Las 10 Heurísticas de Usabilidad de Nielsen:

Las heurísticas de Nielsen (1994) son diez principios de diseño que sirven como lista de chequeo para evaluar si una interfaz está bien construida. Son el estándar más usado en la industria para inspección de usabilidad sin necesitar usuarios reales en el proceso.

#	Heurística	Descripción	Aplicación en TRUEQUE MATCH
1	Visibilidad del estado del sistema	El sistema siempre informa al usuario qué está pasando, sin dejarlo en el limbo.	Badge de notificaciones con número, estados de trueque visibles (pendiente/aceptado/completado), indicadores de carga en botones.
2	Correspondencia sistema-mundo real	El sistema habla el idioma del usuario: palabras cotidianas, no términos técnicos.	Mensajes en español colombiano sin códigos de error. 'Este correo ya está registrado' en lugar de 'Error 409: Duplicate key'.
3	Control y libertad del usuario	El usuario puede deshacer acciones y salir cuando quiera sin consecuencias graves.	Botón de retroceso en todos los flujos, opción de cancelar un trueque, guardar borrador al salir del formulario.
4	Consistencia y estándares	Los mismos elementos se comportan igual en toda la app. Lo que funciona igual, se ve igual.	Mismos colores, tipografía y comportamiento de botones en todas las pantallas. El rojo siempre es acción principal.
5	Prevención de errores	Mejor evitar que el error ocurra que tener que informarlo después.	Botón INGRESAR desactivado hasta que ambos campos sean válidos. Confirmación antes de cancelar un trueque activo.
6	Reconocimiento sobre recuerdo	El usuario no debería tener que memorizar información entre pantallas.	El estado del trueque siempre visible, historial accesible desde el perfil, nombre de la oferta en el chat.
7	Flexibilidad y eficiencia de uso	Atajos para usuarios expertos sin perjudicar a los novatos.	Chips de categorías para filtrado rápido, botón Publicar (+) siempre visible en Tab Bar central.

8	Diseño estético y minimalista	Cada pantalla muestra solo lo necesario. Más información no significa mejor experiencia.	Tarjetas de oferta con info esencial: imagen, título, usuario y categoría. El detalle está en pantalla secundaria.
9	Ayuda a reconocer y recuperarse de errores	Los mensajes de error son comprensibles y ofrecen una solución concreta.	'La cédula solo puede tener números (8 a 10 dígitos)' en lugar de 'Formato inválido'.
10	Ayuda y documentación	Si el usuario necesita ayuda, debe poder encontrarla fácilmente.	Tooltips en primera visita (onboarding), textos de ayuda bajo campos del formulario, sección de preguntas frecuentes.

1.4 Reglas de Usabilidad de Shneiderman

Shneiderman et al. (2016) proponen ocho reglas de oro del diseño de interfaces, complementarias a las heurísticas de Nielsen. Estas reglas surgieron del estudio de sistemas interactivos de alto uso y se centran en la consistencia y el control del usuario.

Regla	Descripción	Ejemplo en TRUEQUE MATCH
1. Consistencia	Usar las mismas secuencias de acciones, terminología idéntica y formato visual uniforme en situaciones similares.	El mismo formulario de campo (etiqueta roja arriba, input oscuro, mensaje de error abajo) en Login, Registro y Publicar oferta.
2. Atajos para usuarios frecuentes	Ofrecer mecanismos de aceleración para usuarios	Chips de categorías para filtrar sin abrir un menú adicional. Publicar visible siempre en Tab Bar.

	expertos sin afectar a los novatos.	
3. Retroalimentación informativa	Toda acción del usuario debe generar una respuesta del sistema, proporcional a la importancia de la acción.	Toast de confirmación al publicar oferta, badge actualizado al llegar notificación, estado del botón que cambia al enviar formulario.
4. Diálogos de cierre	Las secuencias de acciones deben tener inicio, desarrollo y fin claro para dar sensación de logro.	Flujo de registro en 3 pasos con confirmación final: 'Tu cuenta fue creada exitosamente'. Pantalla de evaluación al cerrar un trueque.
5. Prevención y manejo de errores	Diseñar el sistema para que sea difícil cometer errores graves. Si ocurren, ofrecer recuperación simple.	Validación en tiempo real en todos los formularios. Confirmación obligatoria antes de acciones irreversibles.
6. Permitir deshacer acciones	Las acciones deben ser reversibles siempre que sea posible, para reducir la ansiedad del usuario.	Posibilidad de cancelar una propuesta de trueque antes de que sea aceptada. Editar o desactivar una oferta publicada.
7. Sensación de control	El usuario debe sentir que controla el sistema, no al revés. El sistema responde a sus acciones, no las impone.	El usuario decide cuándo publicar, cuándo proponer y cuándo completar un trueque. El sistema no hace nada sin su confirmación.

8. Reducir la carga de memoria a corto plazo	Minimizar la información que el usuario debe recordar entre pantallas. Mostrar lo necesario en el momento justo.	Resumen del trueque visible en el chat, nombre del otro usuario en todas las pantallas del flujo, estado siempre actualizado.
---	--	---

Sección 2. Accesibilidad en Aplicaciones de Software

2.1 ¿Qué es la accesibilidad?

La accesibilidad en software es el diseño y desarrollo de aplicaciones que pueden ser usadas por el mayor número posible de personas, incluyendo aquellas con algún tipo de discapacidad visual, auditiva, motora o cognitiva. No es un extra ni un lujo: es un derecho y, en muchos países, una obligación legal.

W3C (2018) la define como la práctica de hacer que los sitios y aplicaciones web sean utilizables por personas con discapacidades. Pero la accesibilidad también beneficia a personas sin discapacidades: quien usa el celular al sol con brillo reducido agradece el buen contraste; quien tiene el brazo ocupado agradece los botones grandes. Es como la rampa de acceso en un edificio: la necesitan las personas en silla de ruedas, pero la usan también quienes van con carrito de bebé o maleta con ruedas.

2.2 Pautas WCAG 2.1 El Estándar Internacional

Las Web Content Accessibility Guidelines (WCAG 2.1) son el estándar internacional de accesibilidad publicado por el W3C (2018). Se organizan bajo cuatro principios fundamentales, conocidos como POUR:

Principio	Significado	Qué implica
Perceptible	La información debe presentarse de forma que todos los usuarios puedan percibirla.	Textos alternativos en imágenes, subtítulos en videos, contraste de color suficiente entre texto y fondo.
Operable	Los componentes de la interfaz deben poder operarse por todos los usuarios.	Navegación completa por teclado, sin acciones que requieran coordinación física precisa, tiempo suficiente para leer.
Comprensible	La información y el funcionamiento deben ser comprensibles.	Lenguaje claro, comportamiento predecible, ayuda para evitar y corregir errores de entrada.
Robusto	El contenido debe interpretarse correctamente por distintas tecnologías de asistencia.	Código HTML semántico, etiquetas ARIA correctas, compatibilidad con lectores de pantalla como NVDA o VoiceOver.

2.3 Niveles de Conformidad WCAG:

Las pautas WCAG se organizan en tres niveles de cumplimiento:

Nivel	Descripción	Requisito mínimo para TRUEQUE MATCH
A (mínimo)	Criterios básicos que si no se cumplen hacen la app inutilizable para ciertos usuarios.	Textos alternativos en imágenes, sin contenido que parpadee más de 3 veces por segundo.
AA (recomendado)	Nivel estándar en la industria. Es el objetivo de TRUEQUE MATCH.	Contraste mínimo 4.5:1 para texto normal, 3:1 para texto grande. Navegación por teclado completa.
AAA (óptimo)	El nivel más alto. Difícil de cumplir al 100% en todas las páginas.	Contraste 7:1, lenguaje simplificado, sin límite de tiempo en ninguna tarea.

2.4 Reglas de Accesibilidad Aplicadas a TRUEQUE MATCH:

A continuación se describen las reglas de accesibilidad concretas que se aplican en el diseño de TRUEQUE MATCH, cada una con su justificación técnica y ejemplo real:

Regla	Descripción	Aplicación en TRUEQUE MATCH
Contraste de color suficiente	El texto debe tener una relación de contraste mínima de 4.5:1 con el fondo para ser legible por personas con baja visión o daltonismo (WCAG 2.1, criterio 1.4.3).	Texto blanco cálido #F5F0EB sobre fondo oscuro #1A1A1A: contraste aproximado 12:1. Los placeholders se ajustaron de #888 a #AAA para superar el umbral mínimo.

Tamaño de elementos táctiles	Los elementos interactivos en pantallas táctiles deben tener un área mínima de 44×44 píxeles para ser activados con precisión (WCAG 2.1, criterio 2.5.5).	Botones principales de al menos 48 px de alto, ícono de la Tab Bar de 44×44 px mínimo, chips de categorías con padding generoso.
Etiquetas ARIA en formularios	Los campos de formulario deben tener etiquetas descriptivas para que los lectores de pantalla puedan anunciarlos correctamente a usuarios con discapacidad visual.	Atributos aria-label en campos sin etiqueta visible, aria-invalid='true' en campos con error, aria-describedby apuntando al mensaje de error del campo.
Textos alternativos en imágenes	Toda imagen que transmite información debe tener un atributo alt descriptivo para que lectores de pantalla comuniquen su contenido (WCAG 2.1, criterio 1.1.1).	Imágenes de ofertas con alt que describe el contenido: alt='Celular Xiaomi Redmi 10 en buen estado'. Avatares con alt='Foto de perfil de [nombre]'.
Navegación por teclado	Todos los elementos interactivos deben ser accesibles y operables usando solo el teclado, sin depender del ratón ni del toque (WCAG 2.1, criterio 2.1.1).	En la versión web de TRUEQUE MATCH, todos los botones, links y campos son alcanzables con Tab. El foco visible se muestra con borde destacado.

Lenguaje claro y simple	El contenido textual debe redactarse en el nivel de comprensión más bajo posible, evitando tecnicismos innecesarios (WCAG 2.1, criterio 3.1.5).	Mensajes de error en lenguaje cotidiano: 'Debes tener al menos 18 años para usar la app' en lugar de 'Fecha fuera del rango permitido'.
Sin dependencia exclusiva del color	La información no debe transmitirse únicamente por color, ya que los usuarios con daltonismo no podrán percibirla (WCAG 2.1, criterio 1.4.1).	Los campos con error muestran borde rojo Y un ícono  Y texto descriptivo. El estado del trueque usa color Y texto Y ícono simultáneamente.
Compatibilidad con tecnologías asistidas	El código debe seguir estándares semánticos para funcionar correctamente con lectores de pantalla, ampliadores y otros dispositivos de asistencia.	Uso de etiquetas HTML semánticas en la versión web (button, input, label, nav). Roles ARIA donde el HTML no es suficientemente descriptivo.

Sección 3. Tipos de Aplicaciones: Independiente vs. Web

3.1 Aplicación Independiente (Desktop / Nativa)

Una aplicación independiente es un programa que se instala directamente en el dispositivo del usuario (computador, celular o tablet) y puede funcionar total o parcialmente sin conexión a internet. Se ejecuta en el sistema operativo del dispositivo usando sus recursos locales: procesador, memoria y almacenamiento.

El nombre 'independiente' hace referencia a que no depende de un servidor web para funcionar: toda su lógica, o la mayor parte de ella, vive en el dispositivo. Un editor de texto como Microsoft Word, una calculadora o un juego instalado son ejemplos clásicos de aplicaciones independientes.

Características principales

Característica	Descripción
Instalación local	El usuario descarga e instala el programa en su dispositivo. Ocupa espacio en el disco.
Acceso a recursos del sistema	Puede acceder a archivos locales, cámara, GPS, sensores, micrófono y notificaciones push sin restricciones del navegador.
Funcionamiento offline	Opera sin internet, o con conectividad parcial. Sincroniza datos cuando hay conexión disponible.
Mayor rendimiento	Al ejecutarse nativamente en el hardware, tiene acceso directo a la GPU y CPU sin capas intermedias.
Actualizaciones manuales	El usuario debe descargar e instalar las actualizaciones. Puede haber versiones desactualizadas en uso.
Distribución por tiendas	Se distribuye por Google Play Store, Apple App Store o instaladores directos (.exe, .apk, .dmg).
Seguridad del entorno local	Los datos pueden almacenarse localmente con cifrado, sin depender de la seguridad del servidor.

Tipos de aplicaciones independientes:

- Aplicaciones de escritorio nativas: desarrolladas específicamente para un sistema operativo (Windows, macOS, Linux). Ejemplo: Microsoft Excel, Adobe Photoshop.
- Aplicaciones móviles nativas: desarrolladas para Android (Java/Kotlin) o iOS (Swift/Objective-C). Tienen acceso completo a las APIs del sistema operativo.
- Aplicaciones multiplataforma nativas: código único que compila a nativo en varios sistemas operativos. Ejemplos: Electron (escritorio), React Native y Flutter (móvil).

3.2 Aplicación Web:

Una aplicación web es un programa que se ejecuta en un servidor remoto y el usuario accede a ella a través de un navegador web (Chrome, Firefox, Safari). No requiere instalación: con tener acceso a internet y un navegador es suficiente para usarla.

La lógica de la app vive en el servidor, no en el dispositivo del usuario. El navegador actúa como ventana de acceso: muestra la interfaz y envía las acciones del usuario al servidor, que procesa y responde. Correo en Gmail, gestionar una cuenta bancaria online o usar Google Docs son ejemplos de aplicaciones web.

Características principales:

Característica	Descripción
Sin instalación	El usuario accede con URL desde cualquier navegador. Sin descarga ni ocupación de espacio local.
Actualizaciones automáticas	El desarrollador actualiza la app en el servidor y todos los usuarios ven los cambios de inmediato.
Acceso multiplataforma	Funciona en cualquier dispositivo con navegador: computador, celular, tablet, Smart TV.
Dependencia de conexión	Generalmente requiere internet para funcionar. Sin conexión, la app no está disponible.
Acceso limitado al hardware	El navegador restringe el acceso a archivos locales, cámara y sensores por razones de seguridad.
Distribución inmediata	No depende de tiendas de aplicaciones ni procesos de aprobación. El desarrollador publica cuando quiere.
Mantenimiento centralizado	Todos los usuarios usan la misma versión. Más fácil de mantener y corregir errores.

Tipos de aplicaciones web:

- Sitios web estáticos: contenido fijo que no cambia según el usuario. Solo HTML, CSS y JavaScript básico.
- Aplicaciones web dinámicas: el contenido cambia según el usuario, sus datos y sus acciones. Requieren backend y base de datos.

- Progressive Web Apps (PWA): aplicaciones web que se comportan casi como apps nativas: se pueden instalar en la pantalla de inicio, enviar notificaciones y funcionar offline parcialmente.
- Single Page Applications (SPA): cargan una sola página HTML y actualizan el contenido dinámicamente sin recargar el navegador. Ejemplos: Gmail, Trello, React apps.

3.3 Comparación: Aplicación Independiente vs. Aplicación Web

Criterio	App Independiente	App Web
Instalación	Requiere instalación en el dispositivo	Solo necesita un navegador web
Acceso offline	Funciona sin internet (total o parcial)	Generalmente requiere conexión constante
Rendimiento	Mayor: acceso directo al hardware	Menor: capa del navegador como intermediario
Actualizaciones	Manual: el usuario debe actualizar	Automática: el servidor siempre tiene la última versión
Acceso al hardware	Completo: cámara, GPS, archivos, sensores	Limitado por las restricciones de seguridad del navegador
Distribución	App Store, Play Store o instalador directo	URL pública, sin necesidad de aprobación de tiendas

Multiplataforma	Limitada: versión diferente por SO	Alta: un solo código sirve para cualquier dispositivo con navegador
Costo de desarrollo	Mayor: versiones por plataforma	Menor: una sola base de código
Seguridad de datos	Almacenamiento local cifrado	Depende de la seguridad del servidor y la transmisión HTTPS
Experiencia de usuario	Más fluida y nativa	Puede variar según el navegador y la conexión

3.4 Tipo de Aplicación de TRUEQUE MATCH

TRUEQUE MATCH se desarrolla como una aplicación híbrida, lo que significa que combina lo mejor de los dos tipos. Utiliza React Native con Expo para generar una app nativa en Android e iOS (independiente) y una Progressive Web App (PWA) accesible desde el navegador, todo desde un único código base.

Aspecto	Decisión tomada en TRUEQUE MATCH	Justificación
Tipo de app	Híbrida (nativa + PWA)	Un solo código base reduce costos de desarrollo y mantenimiento, clave para un proyecto de bajo presupuesto.
Plataformas objetivo	Android 8+, iOS 13+, navegadores modernos	Cubre el 95%+ de los dispositivos activos en Colombia.

Acceso offline	Parcial: consulta de ofertas guardadas offline con PWA	El usuario puede explorar ofertas sin internet; las acciones de trueque requieren conexión.
Distribución	Play Store, App Store y URL web (PWA)	Múltiples canales de acceso sin costo adicional de desarrollo.
Reglas de usabilidad aplicadas	WCAG 2.1 nivel AA, heurísticas de Nielsen, reglas de Shneiderman	Garantiza que la app sea accesible y fácil de usar para el perfil del usuario colombiano objetivo.

Conclusiones

La usabilidad y la accesibilidad no son opcionales en el desarrollo de software moderno: son la diferencia entre una app que la gente usa y una que desinstala a los dos minutos. Las reglas de Nielsen y Shneiderman, y las pautas WCAG 2.1, proporcionan un marco concreto y medible para construir interfaces que realmente funcionen para el usuario.

Conocer la diferencia entre aplicaciones independientes y aplicaciones web permite tomar decisiones informadas sobre la arquitectura más adecuada para cada proyecto. En el caso de TRUEQUE MATCH, la decisión de desarrollar una app híbrida responde directamente al perfil del usuario colombiano y a las restricciones de presupuesto del proyecto.

Aplicar estas nociones desde el inicio del diseño, antes de escribir código, reduce costos, evita rediseños y garantiza que la app cumpla con los estándares de calidad que los usuarios esperan en el mercado actual.

Referencias

- International Organization for Standardization. (2018). ISO 9241-11:2018: *Ergonomics of human-system interaction Part 11: Usability: Definitions and concepts*. ISO.
- Nielsen, J. (1993). *Usability engineering*. Academic Press.
- Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 152–158). ACM.
<https://doi.org/10.1145/191666.191729>
- Nielsen, J., y Pernice, K. (2010). *Eyetracking web usability*. New Riders.
- Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill Education.
- Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., y Elmqvist, N. (2016). *Designing the user interface: Strategies for effective human-computer interaction* (6.^a ed.). Pearson.
- Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson Education.
- W3C Web Accessibility Initiative. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG21/>
- W3C Web Accessibility Initiative. (2023). *WCAG 2.2*. World Wide Web Consortium.
<https://www.w3.org/TR/WCAG22/>